دانشگاه
خواجه نصیرالدین طوسی
K. N. Toosi University
of Technology

ترم ۱۴۰۳-۱۴۰۴

مینی پروژه سوم هوش مصنوعی مقدماتی



دانشجویان:
آیلار رحیم زاده -فاطمه سادات ساجد
دبیر: دکتر علیاری

https://colab.research.google.com/drive/1jkzPUGI7PLjaqEk67YpjbgaTpG-b1Vc6?usp=sharing

فهرست

سوال اول ۲

بخش اول..... ۲

بخش دوم..... ٤

سوال دوم..... ۱۰

کران مرتبه اول ۱۰

کران مرتبه دوم ۱۲

سوال سوم..... ۱٤

سوال چهارم ۱٦

سوال پنجم..... ۱۹

سوال ششم ۲۲

سوال هفتم..... ۲٤

سوال اول

بخش اول

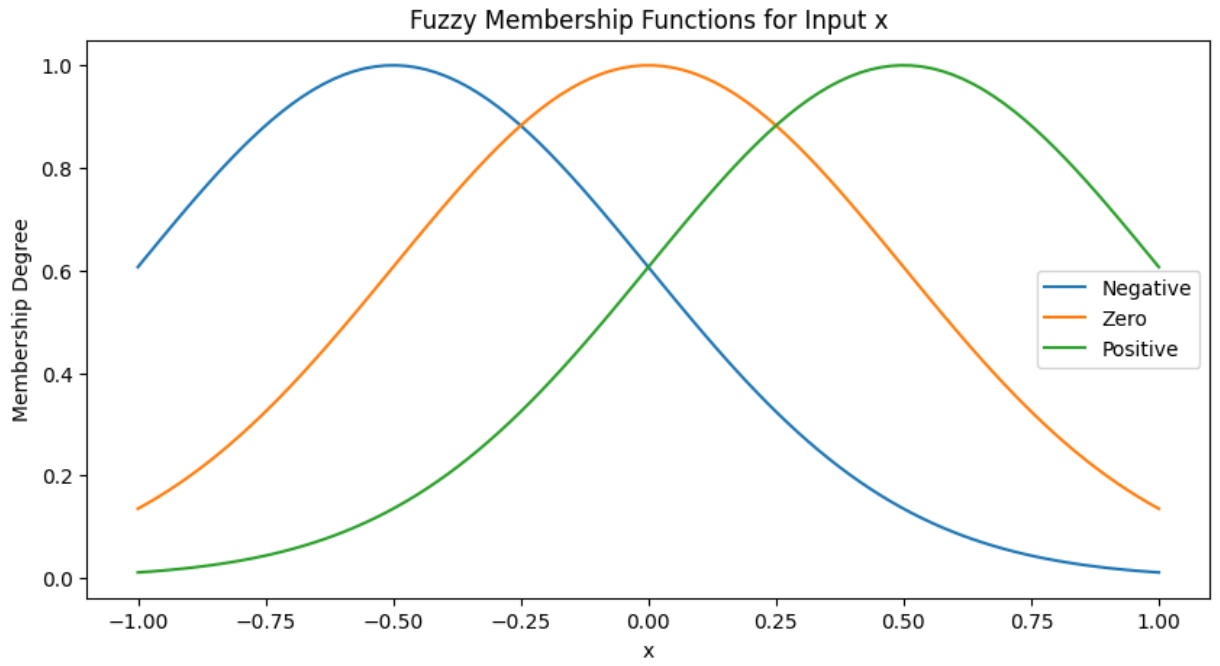
در کد مربوط به سوال، ابتدا تابع h(x) را به صورت دقیق تعریف کرده ایم. سپس به تعریف بازه ای برای x میپردازیم، این محدوده برای اعمال سیستم فازی به کار می‌رود.

```
def h_exact(x):
    return np.exp(-np.pi * x**2) + x * np.sin(np.pi * x)

x_range = np.linspace(-1, 1, 100)
```

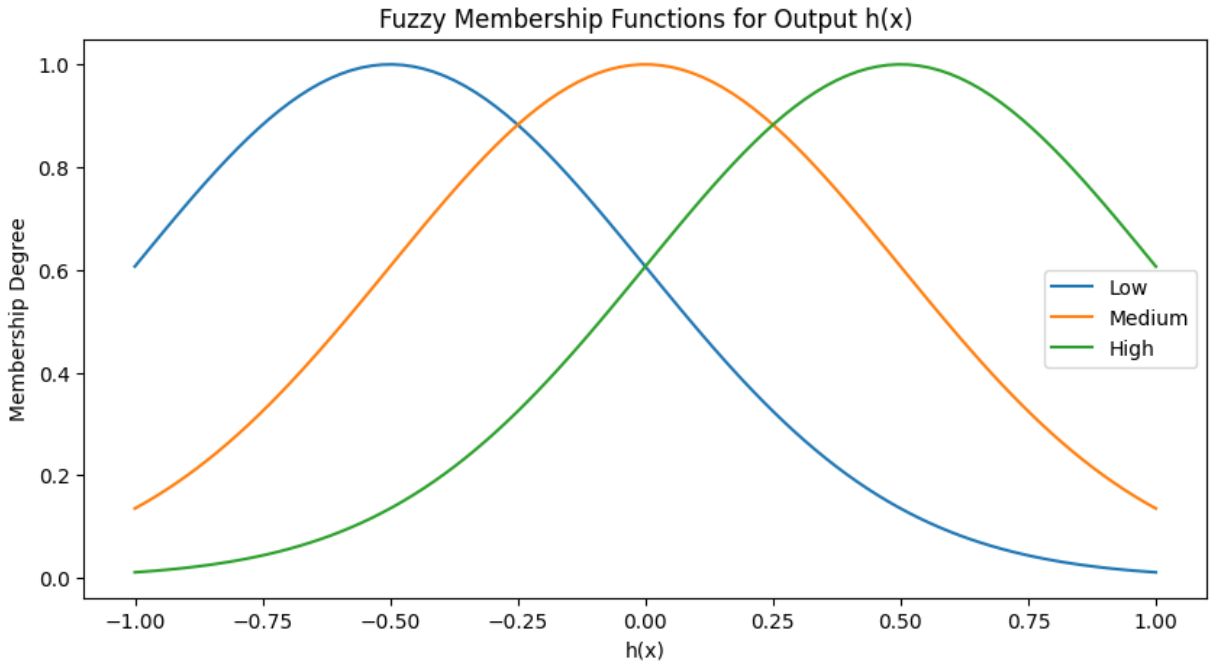
در مرحله بعد از تابع گوسی برای عضویت استفاده کرده‌ایم و یه مجموعه فازی منفی، مثبت، و صفر را در نظر میگیریم. میانگین و انحراف معیار در کد مشخص اند. سپس این توابع عضویت را به نمایش در آورده‌ایم.

```
x_neg = fuzz.gaussmf(x_range, mean=-0.5, sigma=0.5)
x_zero = fuzz.gaussmf(x_range, mean=0, sigma=0.5)
x_pos = fuzz.gaussmf(x_range, mean=0.5, sigma=0.5)
```



برای تابع خروجی، سه تابع عضویت کم، متوسط، و زیاد را در نظر گرفته‌ایم که باز هم توابعی گوسی هستند. این توابع را نیز رسم کرده ایم.

```
h_low = fuzz.gaussmf(x_range, mean=-0.5, sigma=0.5)
h_medium = fuzz.gaussmf(x_range, mean=0, sigma=0.5)
h_high = fuzz.gaussmf(x_range, mean=0.5, sigma=0.5)
```



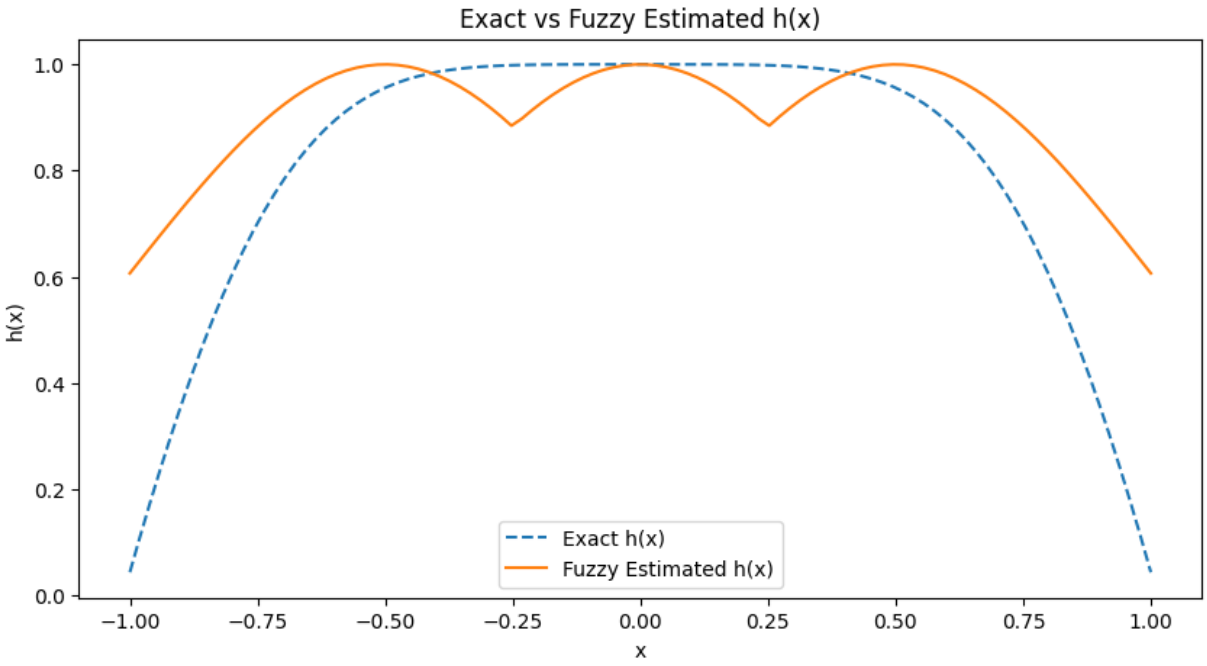
در این مرحله، درجه عضویت هر نقطه را به کمک تابع interp_membership محاسبه میکنیم. برای هر مقدار از دامنه x، درجه عضویت آن در مجموعه‌های فازی مختلف x_{neg}، x_{zero} ، و x_{pos} محاسبه می‌شود.

```
x_neg_degree = fuzz.interp_membership(x_range, x_neg, x_range)
x_zero_degree = fuzz.interp_membership(x_range, x_zero, x_range)
x_pos_degree = fuzz.interp_membership(x_range, x_pos, x_range)
```

در مرحله بعد قوانین فازی به ساده ترین شکل تعریف شده اند و فازی سازی انجام میشود.
در این بخش، از عملگر min برای ترکیب مجموعه‌های فازی ورودی و خروجی استفاده می‌شود. این فرآیند به این معناست که برای هر ورودی، میزان فعالیت در مجموعه‌های فازی خروجی محاسبه می‌شود.

```
low_activation = np.fmin(x_neg_degree, h_low)
medium_activation = np.fmin(x_zero_degree, h_medium)
high_activation = np.fmin(x_pos_degree, h_high)
```

در این مرحله به کمک centroid، سیستم فازی به یک مقدار دقیق و کریسپ (غیر فازی) تبدیل می‌شود. این کار با استفاده از روش مرکز ثقل انجام می‌شود که به معنای محاسبه مرکز وزنی مجموعه‌های فازی است. در نهایت، خطای تخمین خروجی فازی با مقایسه مقدار دقیق تابع h(x) و خروجی تخمینی فازی محاسبه می‌شود.
با توجه به نتایج، دقت کمتر از e=0.1 در بخش هایی از تابع حاصل شده است. برای بهبود نتایج، استفاده از قوانین فازی بیشتر و یا استفاده از توابع عضویت پیچیده تر مانند به کار بردن انحراف معیار پیشنهاد میشود.



بخش دوم

تعریف مجموعه‌های فازی ورودی‌ها (Input Membership Functions):
در مرحله اول مانند قبل تابع را تعریف می‌کنیم سپس مجموعه‌های فازی ورودی‌ها را تعریف می‌کنیم، برای ورودی x، سه مجموعه فازی تعریف شده‌است

- منفی: برای مقادیر منفی از x
- صفر: برای مقادیر نزدیک به صفر
- مثبت: برای مقادیر مثبت از x

این مجموعه‌ها با استفاده از توزیع گوسی (Gaussian) تعریف شده‌اند:

```
input_mfs = {'Negative': (lambda x: gaussian_mf(x, -0.7, 0.3)),
             'Zero': (lambda x: gaussian_mf(x, 0, 0.3)),
             'Positive': (lambda x: gaussian_mf(x, 0.7, 0.3))}
```

حالا مجموعه فازی خروجی‌ها را تعریف می‌کنیم.

```
output_mfs = {'Low': (lambda y: gaussian_mf(y, -0.5, 0.2)),
              'Medium': (lambda y: gaussian_mf(y, 0, 0.2)),
              'High': (lambda y: gaussian_mf(y, 0.5, 0.2))}
```

در این بخش یک سیستم ممدانی به همراه آموزش گرادیان نزولی طراحی میکنیم.

حداقل به سه قانون نیاز داریم اما اینجا برای افزایش عملکرد، تعداد قوانین را به ۹ عدد افزایش داده ایم :

```
rules = [
    ('Negative', 'Low'),
    ('Negative', 'Medium'),
    ('Negative', 'High'),
    ('Zero', 'Low'),
    ('Zero', 'Medium'),
    ('Zero', 'High'),
    ('Positive', 'Low'),
    ('Positive', 'Medium'),
    ('Positive', 'High')
]
```

طبق خواسته سوال، سیصد نقطه یکنواخت به عنوان ورودی تولید کرده ایم. سپس مجموعه های فازی ورودی و خروجی را نوشته ایم.

```
x_vals = np.linspace(-1, 1, 300)
y_vals = h(x_vals)
```

در این قسمت، تابع Fuzzify برای هر مقدار ورودی x، درجات عضویت آن در مجموعه‌های فازی ورودی محاسبه می‌شود. برای هر مجموعه فازی ورودی، یک درجه عضویت محاسبه می‌شود.

```
def fuzzify(x, mfs):
    membership_values = {key: mf(x) for key, mf in mfs.items()}
    return membership_values
```

تابع infer برای استنتاج خروجی با استفاده از قوانین فازی طراحی شده است. در اینجا، ورودی‌ها از طریق مجموعه‌های فازی ورودی فازی می‌شوند و سپس این درجات عضویت برای استنتاج خروجی از طریق قوانین فازی استفاده می‌شوند.

```
def infer(x, input_mfs, output_mfs, rules):
    input_degrees = fuzzify(x, input_mfs)
    rule_outputs = []

    for rule in rules:
        input_set, output_set = rule
        degree = input_degrees[input_set]  # Membership degree for the
input fuzzy set
        rule_output_mf = output_mfs[output_set]  # Output membership
function
        rule_outputs.append(degree * rule_output_mf(x))  # Use input
degree to modulate output

    # Return the sum of rule outputs divided by the sum of input degrees
(normalization)
    return np.sum(rule_outputs) / np.sum(list(input_degrees.values()))
```

برای تبدیل خروجی فازی به یک مقدار دقیق (کریسپ)، از روش centroid استفاده می‌شود. این کار با استفاده از میانگین گرفتن از خروجی‌ها انجام می‌شود.

```
def defuzzify(fuzzy_output):
    return np.sum(fuzzy_output) / len(fuzzy_output)
```

در این قسمت، پارامترهای مجموعه‌های فازی (مانند میانگین‌ها و انحراف معیارها) با استفاده از الگوریتم گرادیان کاهشی به روز می‌شوند. این الگوریتم به تدریج پارامترها را طوری به روز می‌کند که خطای پیش‌بینی کم شود.

```
def gradient_descent(x_vals, y_vals, input_mfs, output_mfs,
learning_rate=0.01):
    for epoch in range(1000):  # Number of iterations
        total_error = 0
        for x, y_true in zip(x_vals, y_vals):
            # Fuzzify the input
            input_degrees = fuzzify(x, input_mfs)
            output_values = []

            # Inference
            for rule in rules:
                input_set, output_set = rule
                degree = input_degrees[input_set]
                output_mf = output_mfs[output_set]
                output_values.append(degree * output_mf(x))

            # Defuzzification
            y_pred = defuzzify(output_values)
            error = (y_true - y_pred)**2
            total_error += error

            # Gradient descent: update parameters
            # For simplicity, we update only means here
            for input_set in input_mfs:
                input_mfs[input_set] =
update_gaussian_params(input_mfs[input_set], learning_rate)

        if epoch % 100 == 0:
            print(f'Epoch {epoch}, Total Error: {total_error}')
```

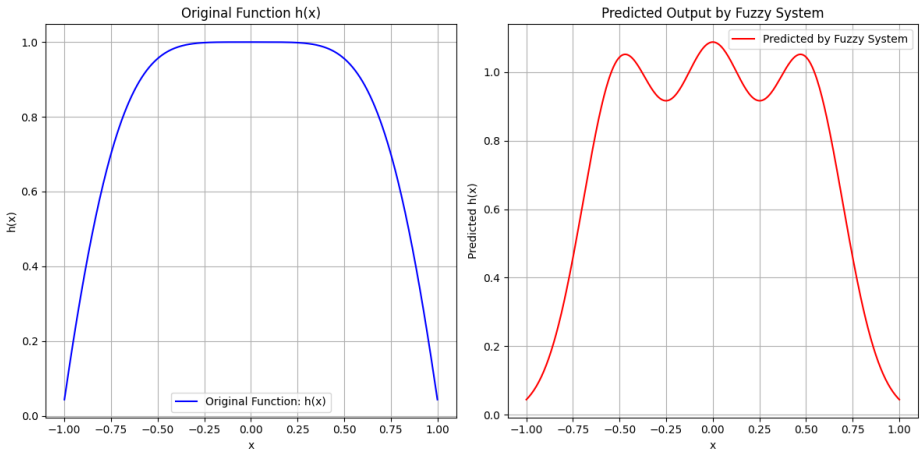
با استفاده از تابع mean-squared-error از کتابخانه sklearn.metrics، خطای مربعی میانگین محاسبه می‌شود تا میزان دقت سیستم فازی بررسی شود.

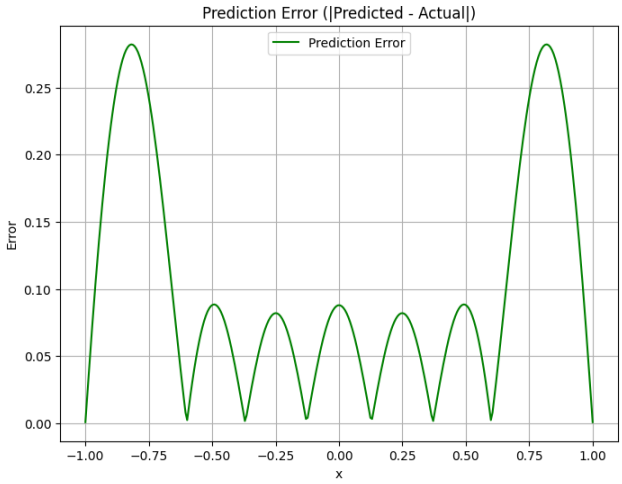
```
predictions = [infer(x, input_mfs, output_mfs, rules) for x in x_vals]
```

```
# Calculate Mean Squared Error
mse = mean_squared_error(y_vals, predictions)
print(f'Mean Squared Error: {mse}')
```

```
Mean Squared Error: 0.017820082356118 75
```

در نهایت، نمودارهای مختلفی برای نمایش تابع اصلی h(x)، پیش‌بینی‌های سیستم فازی و خطای پیش‌بینی ترسیم می‌کنیم.

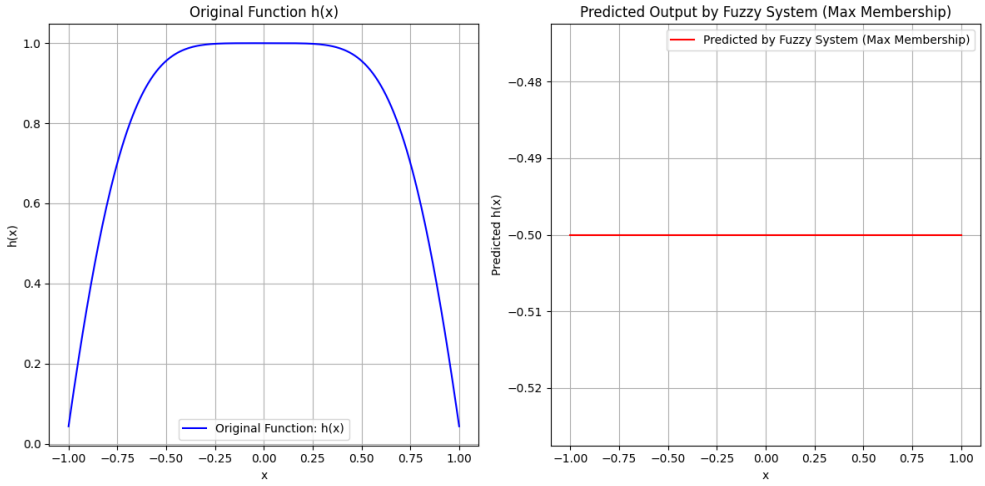


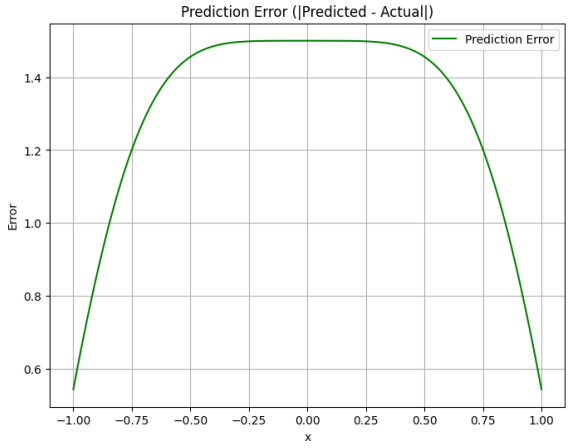


در بلوک های بعدی غیرفازی ساز را به maximum membership ، mean of maxima ، و weighted average تغییر داده ایم. همگی دقت های مشابهی نشان میدهند به غیر از maximum membership که عملکرد ضعیف تری نشان میدهد.

Maximum Membership

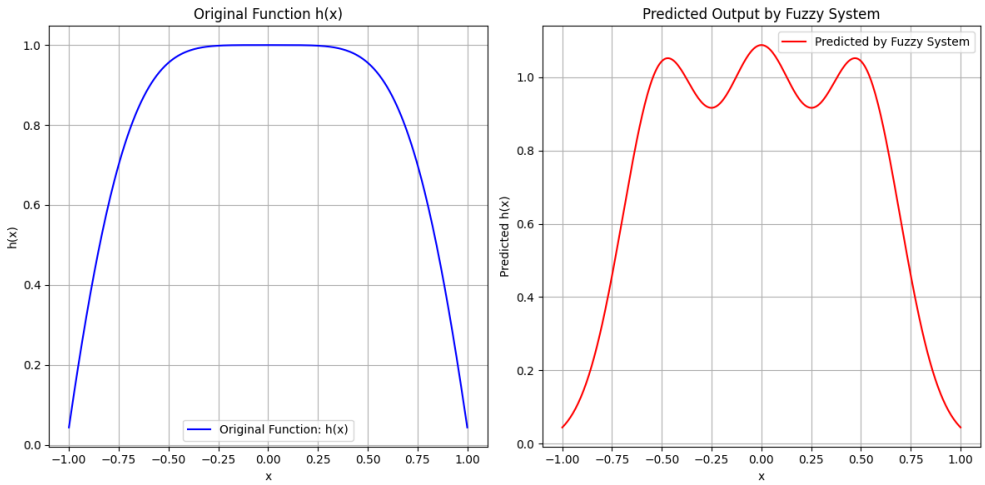
```
Mean Squared Error: 1.785209087669845
```

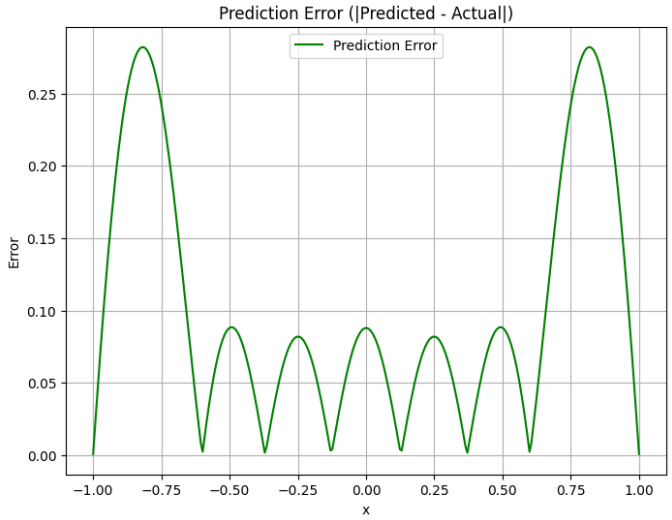




Mean of maxima

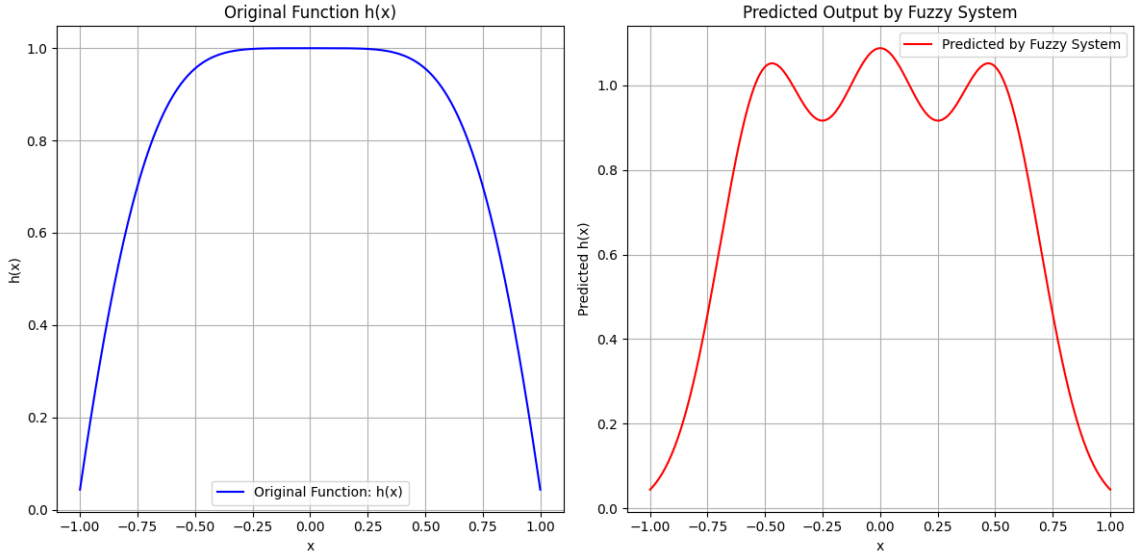
```
Mean Squared Error: 0.017820082356118 75
```

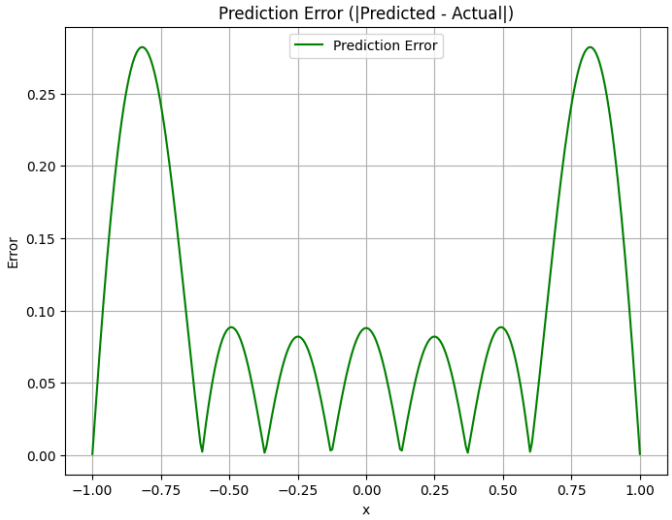




Weighted average

```
Mean Squared Error: 0.01782008235611875
```





سوال دوم

کران مرتبه اول

ابتدا به طراحی بر اساس کران مرتبه اول میپردازیم.
در ابتدا تابع $g(x_1, x_2)$که باید تخمین زده شود، تعریف می‌شود.

```
def g(x1, x2):
    return 1 / (3 + x1 + x2)
```

در این قسمت، برای هر ورودی x_1 و x_2 و همچنین برای خروجی، توابع عضویت مثلثی تعریف می‌شوند.
سه مجموعه فازی برای x_1 و x_2 تعریف شده‌اند (کم، متوسط، زیاد):

```
x1_lo = fuzz.trimf(x1, [-1, -1, 0])
x1_md = fuzz.trimf(x1, [-1, 0, 1])
x1_hi = fuzz.trimf(x1, [0, 1, 1])

x2_lo = fuzz.trimf(x2, [-1, -1, 0])
x2_md = fuzz.trimf(x2, [-1, 0, 1])
x2_hi = fuzz.trimf(x2, [0, 1, 1])
```

برای خروجی نیز به همین صورت تعریف می‌کنیم.

```
g_lo = fuzz.trimf(output, [0, 0, 0.5])
g_md = fuzz.trimf(output, [0, 0.5, 1])
g_hi = fuzz.trimf(output, [0.5, 1, 1])
```

در این مرحله، سه قانون فازی برای ترکیب ورودی‌ها و تولید خروجی تعریف کرده‌ایم. این قوانین به صورت ضرب روابط عضویت ورودی‌ها برای تولید درجات عضویت خروجی‌ها استفاده می‌شوند.

```
rule1 = fuzz.relation_product(x1_lo, x2_lo)
rule2 = fuzz.relation_product(x1_md, x2_md)
rule3 = fuzz.relation_product(x1_hi, x2_hi)
```

تابع fuzzy_inference برای استنتاج خروجی از سیستم فازی تعریف شده است. در اینجا، ابتدا ورودی‌ها فازی می‌شوند (مقادیر درجات عضویت برای ورودی‌ها محاسبه می‌شوند) و سپس با ترکیب درجات عضویت، نتیجه نهایی تخمین زده می‌شود.

```
def fuzzy_inference(x1_value, x2_value):
    # Fuzzify inputs
    x1_lo_degree = fuzz.interp_membership(x1, x1_lo, x1_value)
    x1_md_degree = fuzz.interp_membership(x1, x1_md, x1_value)
    x1_hi_degree = fuzz.interp_membership(x1, x1_hi, x1_value)

    x2_lo_degree = fuzz.interp_membership(x2, x2_lo, x2_value)
    x2_md_degree = fuzz.interp_membership(x2, x2_md, x2_value)
    x2_hi_degree = fuzz.interp_membership(x2, x2_hi, x2_value)
    # Combine input degrees
    combined_x1_x2_lo = min(x1_lo_degree, x2_lo_degree)
    combined_x1_x2_md = min(x1_md_degree, x2_md_degree)
    combined_x1_x2_hi = min(x1_hi_degree, x2_hi_degree)

    # Compute output (fuzzy rule conclusion)
    output_lo = fuzz.defuzz(output, g_lo, 'centroid')
    output_md = fuzz.defuzz(output, g_md, 'centroid')
    output_hi = fuzz.defuzz(output, g_hi, 'centroid')

    # Calculate fuzzy output using the rule degrees
    final_output = (combined_x1_x2_lo * output_lo + combined_x1_x2_md *
output_md + combined_x1_x2_hi * output_hi)

    return final_output
```

در این قسمت، برای مجموعه‌ای از مقادیر ورودی x_1 و x_2، خروجی‌های سیستم فازی را محاسبه می‌کنیم.

```
x1_values = np.linspace(-1, 1, 10)
x2_values = np.linspace(-1, 1, 10)
estimated_values = np.zeros((len(x1_values), len(x2_values)))

for i, x1_val in enumerate(x1_values):
    for j, x2_val in enumerate(x2_values):
        estimated_values[i, j] = fuzzy_inference(x1_val, x2_val)
```

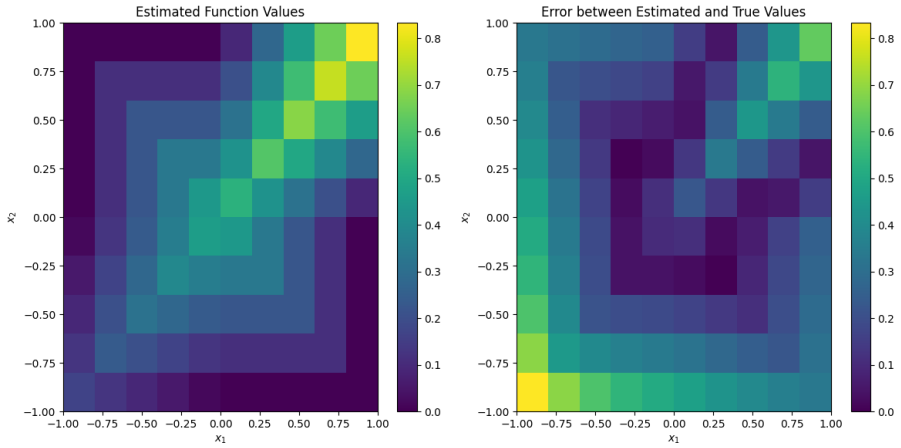
برای مقایسه با خروجی سیستم فازی، مقادیر دقیق تابع $g(x_1, x_2)$ برای هر جفت از مقادیر ورودی را محاسبه می‌کنیم.

```
true_values = np.zeros((len(x1_values), len(x2_values)))
for i, x1_val in enumerate(x1_values):
    for j, x2_val in enumerate(x2_values):
        true_values[i, j] = g(x1_val, x2_val)
```

در این بخش به محاسبه خطا می‌پردازیم.

```
error = np.abs(estimated_values - true_values)
```

در نهایت نمودار مقادیر تخمینی و نمودار خطا را رسم می‌کنیم.



در این مرحله به طراحی غیر فازی ساز با روش میانگین پرداخته ایم. با توجه به شکل نتایج، در برخی نقاط موفق به ثبت خطای کمتر از e=0.1 شده ایم.

کران مرتبه دوم

در ابتدا تابع $g(x_1, x_2) = \frac{1}{3+x_1+x_2}$ به صورت یک تابع ساده دو متغیره تعریف می‌شود.

```
def g(x1, x2):
    return 1 / (3 + x1 + x2)
```

برای ورودی‌های x_1 و x_2، محدوده‌ای از مقادیر از ۱- تا ۱ به تعداد ۱۰۰ مقدار ایجاد می‌کنیم.

```
x1 = np.linspace(-1, 1, 100)
x2 = np.linspace(-1, 1, 100)
```

در اینجا، از توابع عضویت مثلثی برای ورودی‌های x_1 و x_2 استفاده می‌کنیم. توابع عضویت مثلثی، درجه عضویت هر ورودی را در یک مجموعه فازی مشخص می‌کنند.

```
x1_range = fuzz.trimf(x1, [-1, 0, 1])  # Triangular membership function
for x1
x2_range = fuzz.trimf(x2, [-1, 0, 1])  # Triangular membership function
for x2
```

در این مرحله، مقادیر تابع $g(x_1, x_2)$ برای هر جفت مقدار از x_1 و x_2 محاسبه می‌شود. این مقادیر به صورت یک ماتریس دوبعدی ذخیره می‌شوند که هر خانه آن نشان‌دهنده مقدار $g(x_1, x_2)$ برای جفت مربوطه از ورودی‌ها است.

```
g_values = np.array([[g(x1_val, x2_val) for x2_val in x2] for x1_val in
x1])
```

برای فازی سازی از روش "Maximum Membership Defuzzifier" استفاده می‌کنیم. این روش به مقداری را که بیشترین درجه عضویت را دارد به عنوان خروجی انتخاب می‌کند.

```
def max_membership_defuzzifier(membership_values):
    # The defuzzification method is simply taking the value with the
highest membership
    return np.max(membership_values)
```

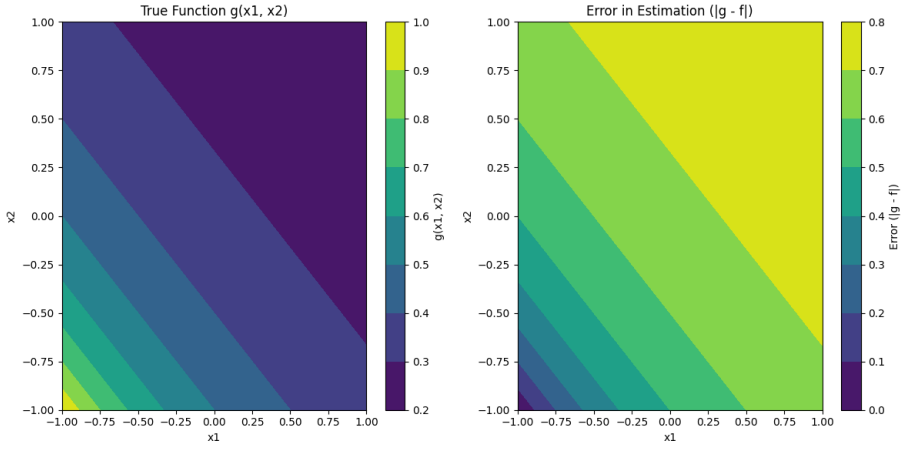
در قسمت، مقادیر خروجی فازی برای هر ترکیب از ورودی‌ها محاسبه می‌شود. در واقع، برای هر ترکیب از x_1 و x_2، درجه عضویت مربوط به آن ترکیب محاسبه شده و سپس خروجی فازی تخمین زده می‌شود.

```
output_values = np.array([[max_membership_defuzzifier(g_values) for _ in
x2] for _ in x1])
```

سپس، خطای تخمینی محاسبه می‌شود. این خطا به صورت تفاوت مطلق بین مقادیر دقیق تابع$g(x_1, x_2)$ و مقادیر تخمینی خروجی فازی محاسبه می‌شود. این مقادیر در یک ماتریس ذخیره می‌شوند.

```
error_values = np.abs(g_values - output_values)
```

نمودار تابع دقیق $g(x_1, x_2)$: این نمودار مقدار دقیق تابع $g(x_1, x_2)$ را برای هر ترکیب از x_1 و x_2 نشان می‌دهد.
نمودار خطا: این نمودار خطای تخمینی (تفاوت بین خروجی فازی و مقدار دقیق) را برای هر ترکیب از x_1 و x_2 نشان می‌دهد.



با توجه به شکل ها، دقت در کران مرتبه دوم در نقاط نزدیک به مقادیر منفی بهتر است اما در مجموع کران مرتبه دوم و روش ماکزیمم دقت کمتری را نشان میدهد.

سوال سوم

در ابتدا پارامتر های مدل Mackey-Glas را تعیین کرده ایم.

```
tau = 17
beta = 0.2
gamma = 0.1
n = 10
x0 = 1.2
steps = 1000
```

با استفاده از معادله Mackey-Glass، مقادیر سری زمانی واقعی x(t) برای ۱۰۰۰ گام زمانی محاسبه می‌شود. برای گام‌های اولیه از مقدار اولیه x_0 استفاده می‌شود، و سپس برای باقی گام‌ها از معادله سیستم استفاده می‌کنیم.

```
x_true = np.zeros(steps)
x_true[:n] = x0

# Mackey-Glass equation to generate true values
for t in range(n, steps):
    x_true[t] = x_true[t-1] + (beta * x_true[t-tau] / (1 + x_true[t-
tau]**n) - gamma * x_true[t-1])
```

این معادله برای هر گام t با توجه به مقادیر قبلی $x(t-1)$ و $x(t-\tau)$ مقدار جدید x(t) را محاسبه می‌کند. بدین شکل مقادیر واقعی برای تخمین ما از طریق معادله مربوطه به دست میایند. سپس دو ورودی فازی سیستم که x و x_tau هستند را به کمک مجموعه های فازی مدلسازی کرده ایم.

```
x_prev = ctrl.Antecedent(np.linspace(0, 2, 100), 'x_prev')  # previous
value of x
x_tau = ctrl.Antecedent(np.linspace(0, 2, 100), 'x_tau')  # value of x(t-
tau)
```

برای هر ورودی سه مجموعه ی low medium high را درنظر گرفته ایم.

```
x_prev['low'] = fuzz.trimf(x_prev.universe, [0, 0, 1])
x_prev['medium'] = fuzz.trimf(x_prev.universe, [0, 1, 2])
x_prev['high'] = fuzz.trimf(x_prev.universe, [1, 2, 2])

x_tau['low'] = fuzz.trimf(x_tau.universe, [0, 0, 1])
x_tau['medium'] = fuzz.trimf(x_tau.universe, [0, 1, 2])
x_tau['high'] = fuzz.trimf(x_tau.universe, [1, 2, 2])
```

سپس مجموعه های فازی خروجی را به کمک توابع عضویت مثلثی تعریف کرده ایم.

```
x_est = ctrl.Consequent(np.linspace(0, 2, 100), 'x_est')  # estimated
value of x
```

```
# Define fuzzy sets for the output
x_est['low'] = fuzz.trimf(x_est.universe, [0, 0, 1])
x_est['medium'] = fuzz.trimf(x_est.universe, [0, 1, 2])
x_est['high'] = fuzz.trimf(x_est.universe, [1, 2, 2])
```

در ادامه ۹ قانون فازی تعریف کرده ایم و تخمین را انجام داده ایم.

```
rule1 = ctrl.Rule(x_prev['low'] & x_tau['low'], x_est['low'])
rule2 = ctrl.Rule(x_prev['low'] & x_tau['medium'], x_est['low'])
rule3 = ctrl.Rule(x_prev['low'] & x_tau['high'], x_est['medium'])

rule4 = ctrl.Rule(x_prev['medium'] & x_tau['low'], x_est['low'])
rule5 = ctrl.Rule(x_prev['medium'] & x_tau['medium'], x_est['medium'])
rule6 = ctrl.Rule(x_prev['medium'] & x_tau['high'], x_est['high'])

rule7 = ctrl.Rule(x_prev['high'] & x_tau['low'], x_est['medium'])
rule8 = ctrl.Rule(x_prev['high'] & x_tau['medium'], x_est['high'])
rule9 = ctrl.Rule(x_prev['high'] & x_tau['high'], x_est['high'])
```

در این مرحله، سیستم فازی با استفاده از قوانین تعریف‌شده ساخته می‌شود و سپس شبیه‌سازی برای پیش‌بینی سری زمانی انجام می‌شود.

```
system = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5, rule6,
rule7, rule8, rule9])
simulator = ctrl.ControlSystemSimulation(system)
```

در اینجا، سیستم فازی برای هر گام زمانی پیش‌بینی می‌کند. ورودی‌های سیستم فازی مقدار $x(t-1)$ و $x(t-\tau)$ هستند و خروجی سیستم مقدار تخمینی $x(t)$ می‌باشد.

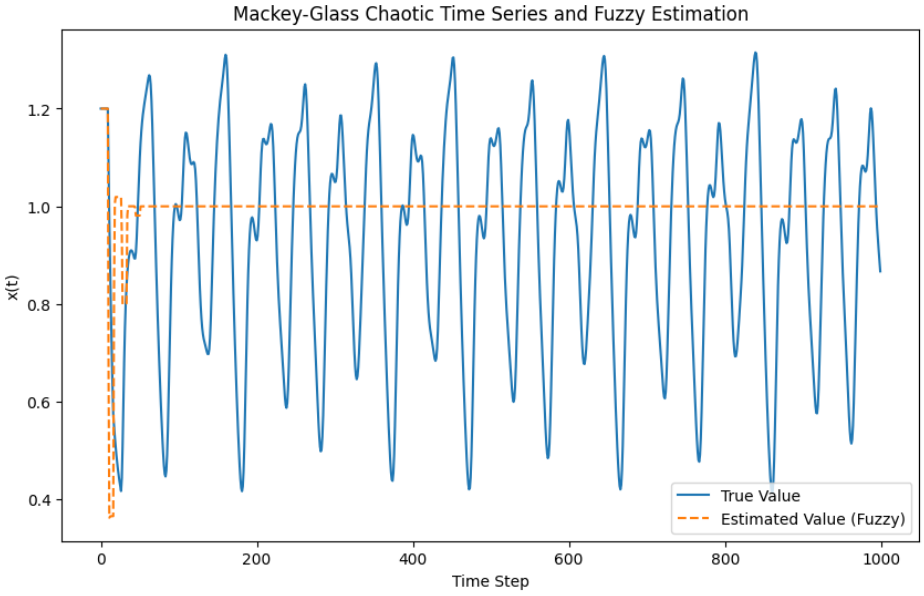
```
# Initialize the estimated time series
x_estimated = np.zeros(steps)
x_estimated[:n] = x0

# Run the fuzzy system to estimate the time series
for t in range(n, steps):
    # Set input values for the fuzzy system
    simulator.input['x_prev'] = x_estimated[t-1]  # previous value
    simulator.input['x_tau'] = x_estimated[t-tau]  # value at t-tau

    # Compute the fuzzy output (estimate)
    simulator.compute()

    # Store the estimated value
    x_estimated[t] = simulator.output['x_est']
```

در نهایت، سری زمانی واقعی و تخمینی را ترسیم می‌کنیم تا مقایسه‌ای بین نتایج واقعی و تخمینی مشاهده شود.
نتایج در زیر نشان دهنده این هستند که در گام های ابتدایی و در x های کوچک، تا حدی موفق به تخمین مقدار اصلی تابع بوده ایم. در ادامه اما مدل به درستی کار نمیکند.



سوال چهارم

پس از فرخوانی کتابخانه ها، تابع g(u) را تعریف کرده ایم.

```
def g(u):
    return 0.6 * np.sin(np.pi * u) + 0.3 * np.sin(3 * np.pi * u) + 0.1 *
np.sin(5 * np.pi * u)
```

سپس داده های ورودی را تولید میکنیم و مقداری دقیق این تابع را به ازای این ورودی ها مشخص میکنیم.

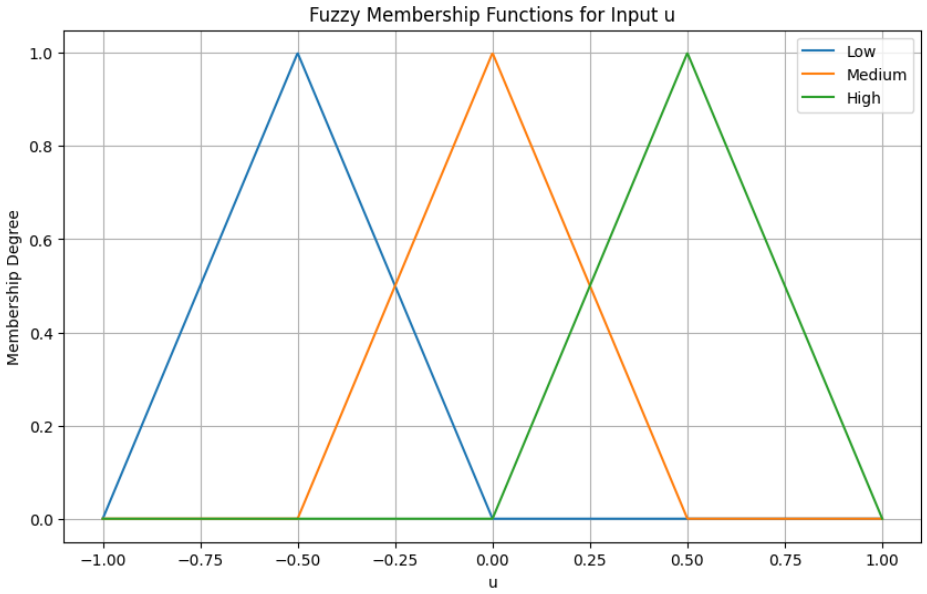
```
u = np.linspace(-1, 1, 1000)  # input range

# Exact g(u)
g_exact = g(u)
```

در این بخش از کد، سه مجموعه فازی برای ورودی u تعریف می‌کنیم. توابع عضویت مثلثی برای این سه مجموعه به شرح زیر تعریف شده‌اند.

```
u_low = fuzz.trimf(u, [-1, -0.5, 0])  # Low membership
u_mid = fuzz.trimf(u, [-0.5, 0, 0.5])  # Medium membership
u_high = fuzz.trimf(u, [0, 0.5, 1])  # High membership
```

سپس این سه تابع مثلثی را رسم کرده ایم.



در ادامه درجه عضویت را به شکل low mid high تعریف میکنیم.

```
def fuzzy_rules(u_val, params):
    low = fuzz.interp_membership(u, u_low, u_val)
    mid = fuzz.interp_membership(u, u_mid, u_val)
    high = fuzz.interp_membership(u, u_high, u_val)

    # Fuzzy rule weighted by learned parameters
    return params[0] * low + params[1] * mid + params[2] * high
```

در قسمت بعدی گرادیان نزولی را پیاده سازی کرده ایم. برای تسریع همگرایی از مومنتوم استفاده شده و شرط توقف زودهنگام نیز اعمال شده است.

```
def gradient_descent(learning_rate, epochs, u_vals, g_exact_vals,
max_gradient=10, momentum=0.9):
    # Initialize fuzzy rule weights (random start for demonstration)
    params = np.random.rand(3)  # 3 parameters: low, mid, high weights
    prev_gradients = np.zeros(3)  # For momentum

    # Gradient descent loop
    for epoch in range(epochs):
        gradients = np.zeros(3)
        total_error = 0

        # Vectorized approach to calculate the error and gradients
        low_vals = fuzz.interp_membership(u, u_low, u_vals)
        mid_vals = fuzz.interp_membership(u, u_mid, u_vals)
        high_vals = fuzz.interp_membership(u, u_high, u_vals)

        # Compute predicted values using current parameters
```

```
        predicted = params[0] * low_vals + params[1] * mid_vals +
params[2] * high_vals

        # Calculate error
        error = g_exact_vals - predicted
        total_error = np.sum(error ** 2)  # Sum of squared errors

        # Compute gradients for each parameter (simplified for this
example)
        gradients[0] = -2 * np.sum(error * low_vals)  # gradient for low
parameter
        gradients[1] = -2 * np.sum(error * mid_vals)  # gradient for mid
parameter
        gradients[2] = -2 * np.sum(error * high_vals) # gradient for high
parameter

        # Clip gradients to avoid overflow and instability
        gradients = np.clip(gradients, -max_gradient, max_gradient)

        # Apply momentum
        gradients = momentum * prev_gradients + (1 - momentum) * gradients

        # Update parameters
        params -= learning_rate * gradients

        # Early stopping condition if the error converges
        if epoch % 50 == 0 and total_error < 1e-5:
            print(f"Converged at epoch {epoch} with total error
{total_error}")
            break

        # Print error for every epoch or when significant change happens
        if epoch % 100 == 0 or abs(total_error - prev_error) > 1e-3:
            print(f'Epoch {epoch}, Total Error: {total_error}')

        # Store previous error for comparison
        prev_error = total_error

        # Adjust learning rate based on error decrease rate
        if epoch > 0 and total_error < prev_error:
            learning_rate *= 0.95  # Reduce learning rate by 5% each time
the error decreases
        # Store gradients for momentum update
        prev_gradients = gradients
```

```
    return params
```

در بخش بعد برای یادگیری ورودی ها و تابع گرادیان فراخوانده میشوند.
در آخر نیز نتایج را نمایش میدهیم.

```
u_vals = np.linspace(-1, 1, 1000)
g_exact_vals = g(u_vals)

# Apply gradient descent
learning_rate = 0.01  # Start with a slightly higher learning rate
epochs = 60
final_params = gradient_descent(learning_rate, epochs, u_vals,
g_exact_vals)

print("Final learned parameters:", final_params)
```

با توجه به خروجی، پس از ۶۰ ایپاک کاهش چشمگیری در خطا مشاهده میشود و مقادیر نهایی یادگیری شده عبارتند از :

[-0.68958737 -0.01585948 0.68350819]

سوال پنجم

در ابتدا، ورودی‌ها و خروجی‌ها برای سیستم فازی تعریف می‌کنیم.

- ورودی‌ها x_1 و x_2 هر دو در بازه $[-1,1]$ با گام‌های ۰٫۱ قرار دارند.
- خروجی در بازه $[0, 2]$ با گام‌های ۰٫۱ قرار دارد.

```
• x1 = np.linspace(-1, 1, 100)
• x2 = np.linspace(-1, 1, 100)
•
• # Define the fuzzy system variables
• x1_input = ctrl.Antecedent(np.arange(-1, 1.1, 0.1), 'x1')
• x2_input = ctrl.Antecedent(np.arange(-1, 1.1, 0.1), 'x2')
• output = ctrl.Consequent(np.arange(0, 2.1, 0.1), 'output')
```

برای هر ورودی و خروجی، توابع عضویت فازی تعریف شده است. برای ورودی‌ها x_1 و x_2 از توابع عضویت مثلثی و مستطیلی (Trapezoidal) استفاده شده است، خروجی نیز از دو تابع عضویت مستطیلی با سطوح "کم" و "زیاد" تشکیل شده است.

```
x1_input['low'] = fuzz.trapmf(x1_input.universe, [-1, -1, -0.5, 0])
x1_input['medium'] = fuzz.trimf(x1_input.universe, [-0.5, 0, 0.5])
x1_input['high'] = fuzz.trapmf(x1_input.universe, [0, 0.5, 1, 1])

x2_input['low'] = fuzz.trapmf(x2_input.universe, [-1, -1, -0.5, 0])
```

```
x2_input['medium'] = fuzz.trimf(x2_input.universe, [-0.5, 0, 0.5])
x2_input['high'] = fuzz.trapmf(x2_input.universe, [0, 0.5, 1, 1])

# Define the output fuzzy membership functions
output['low'] = fuzz.trapmf(output.universe, [0, 0, 0.5, 1])
output['high'] = fuzz.trapmf(output.universe, [1, 1.5, 2, 2])
```

در اینجا، مجموعه‌ای از قوانین فازی تعریف کردیم که به ترکیب ورودی‌ها و تعیین خروجی مرتبط می‌شوند.

```
rule1 = ctrl.Rule(x1_input['low'] & x2_input['low'], output['low'])
rule2 = ctrl.Rule(x1_input['low'] & x2_input['medium'], output['low'])
rule3 = ctrl.Rule(x1_input['low'] & x2_input['high'], output['high'])
rule4 = ctrl.Rule(x1_input['medium'] & x2_input['low'], output['low'])
rule5 = ctrl.Rule(x1_input['medium'] & x2_input['medium'], output['low'])
rule6 = ctrl.Rule(x1_input['medium'] & x2_input['high'], output['high'])
rule7 = ctrl.Rule(x1_input['high'] & x2_input['low'], output['high'])
rule8 = ctrl.Rule(x1_input['high'] & x2_input['medium'], output['high'])
rule9 = ctrl.Rule(x1_input['high'] & x2_input['high'], output['high'])
```

با استفاده از قوانین فازی، یک سیستم فازی می‌سازیم و سپس برای هر ترکیب از x_1 و x_2، شبیه‌سازی انجام می‌شود تا خروجی فازی محاسبه شود.

```
fuzzy_system = ctrl.ControlSystem([rule1, rule2, rule3, rule4, rule5,
rule6, rule7, rule8, rule9])
fuzzy_controller = ctrl.ControlSystemSimulation(fuzzy_system)
```

در این بخش، برای هر ترکیب از x_1 و x_2 در ماتریس 100×100، خروجی سیستم فازی محاسبه می‌شود و در خروجی شبکه ذخیره می‌شود.

```
output_grid = np.zeros((100, 100))
for i in range(100):
    for j in range(100):
        fuzzy_controller.input['x1'] = x1[i]
        fuzzy_controller.input['x2'] = x2[j]
        fuzzy_controller.compute()
        output_grid[i, j] = fuzzy_controller.output['output']
```

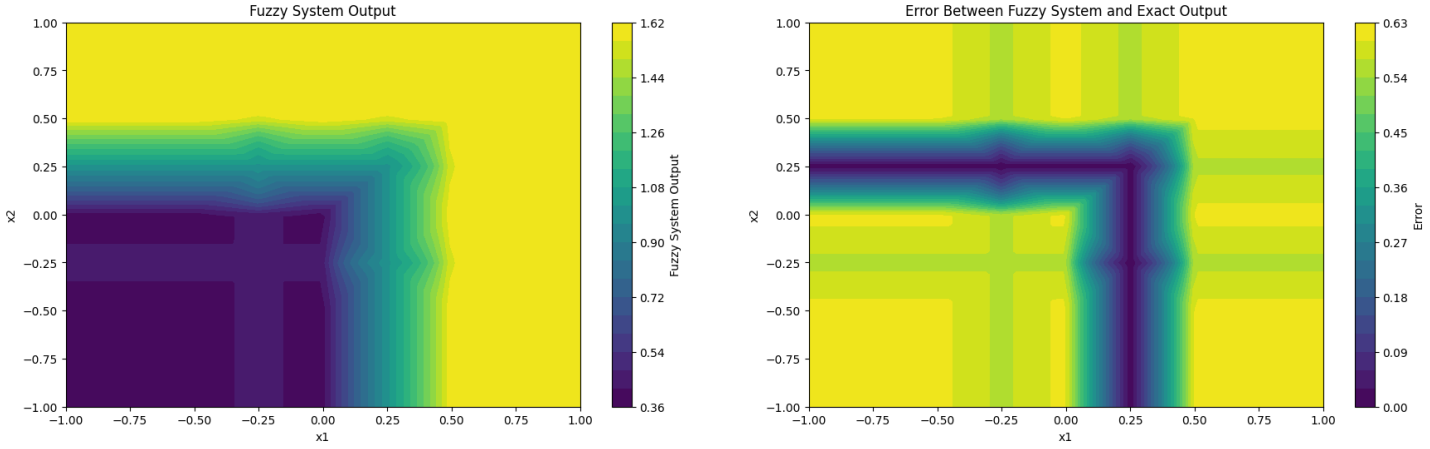
در اینجا، یک خروجی به عنوان مرجع برای مقایسه با خروجی سیستم فازی تعریف می‌شود.

```
exact_output = np.ones_like(output_grid)
```

محاسبه خطا

```
error = np.abs(output_grid - exact_output)
```

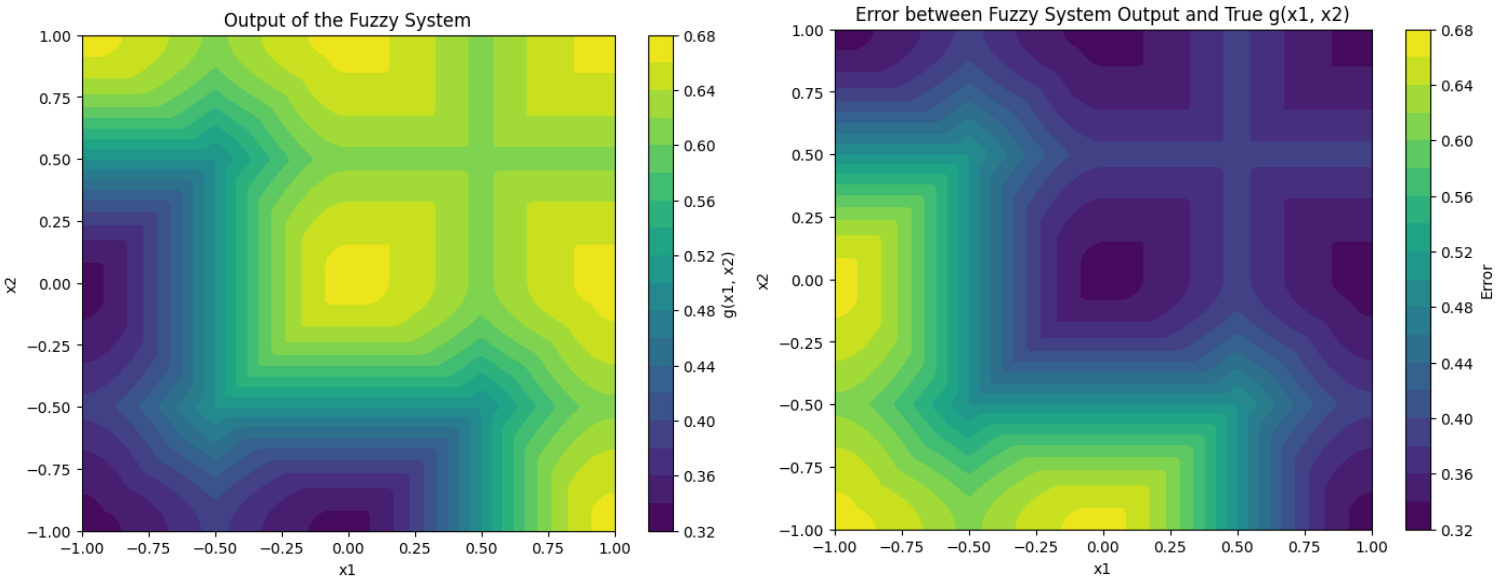
در نهایت نمودار خروجی و نمودار خطا را رسم می‌کنیم.



الف) خروجی و دقت سیستم با کران مرتبه اول و غیر فازی ساز centroid

برای مرتبه دوم همان مراحل را طی می‌کنیم ولی متغیرهای ورودی رو با عضـــویت مثلثی در نظر می‌گیریم.

سیستم با کران مرتبه دوم در زیر نشان داده شده‌است.



ب) خروجی و دقت سیستم با کران مرتبه دوم و غیر فازی ساز centroid

با توجه به شکل ها، نواحی بروز خطا تقریبا مکمل هستند اما در مجموع سیستم با کران مرتبه دوم خطای کمتر از خود نشان میدهد و تخمین دقیق تری دارد. غیرفازی ساز به شکل centroid طراحی شده است.

سوال ششم

۱. طراحی مجموعه های فازی مناسب برای ورودی و خروجی :
مجموعه های ورودی :

Distance ($\dot{d}$) fuzzy sets:

We can define three fuzzy sets for distance:

- **Close**: The robot is near the goal.
- **Medium**: The robot is at a moderate distance from the goal.
- **Far**: The robot is far from the goal.

Angle difference (α) fuzzy sets:

For angle difference, we can have five fuzzy sets:

- **Negative Large (NL)**: Large negative angle error.
- **Negative Small (NS)**: Small negative angle error.
- **Zero (Z)**: No angular error (straight ahead).
- **Positive Small (PS)**: Small positive angle error.
- **Positive Large (PL)**: Large positive angle error.

مجموعه های خروجی :

The linear velocity should be in the range $0 \leq v \leq 0.5$, and we can define:

- **Low (L)**: Small speed.
- **Medium (M)**: Moderate speed.
- **High (H)**: Maximum speed.

Angular velocity (ω) fuzzy sets:

The angular velocity is constrained by $-1 \leq \omega \leq 1$, so we can define:

- **Negative (N)**: Turning left.
- **Zero (Z)**: No rotation (straight ahead).
- **Positive (P)**: Turning right.

۲. طراحی قواعد فازی :
ایده پشت طراحی این قواعد بدین گونه است که زمانی که ربات دور از هدف حرکت میکند، سریع تر مسیر خود را بپیماید و زاویه خود را نیز اصلاح کند. زمانی که نزدیک به هدف میشود، سرعت خود را کم کند و نسبت به اصلاح زاویه حساس تر عمل کند. یک مثال برای این طراحی را در ادامه مشاهده میکنید :

Distance (d)	Angle (α)	Linear velocity (v)	Angular velocity (ω)
Close	NL	High	Positive (P)
Close	NS	Medium	Zero (Z)
Close	Z	Low	Zero (Z)
Medium	NL	Medium	Positive (P)
Medium	Z	Medium	Zero (Z)
Medium	PS	Medium	Negative (N)
Far	NL	Medium	Positive (P)
Far	Z	Medium	Zero (Z)
Far	PL	Low	Negative (N)

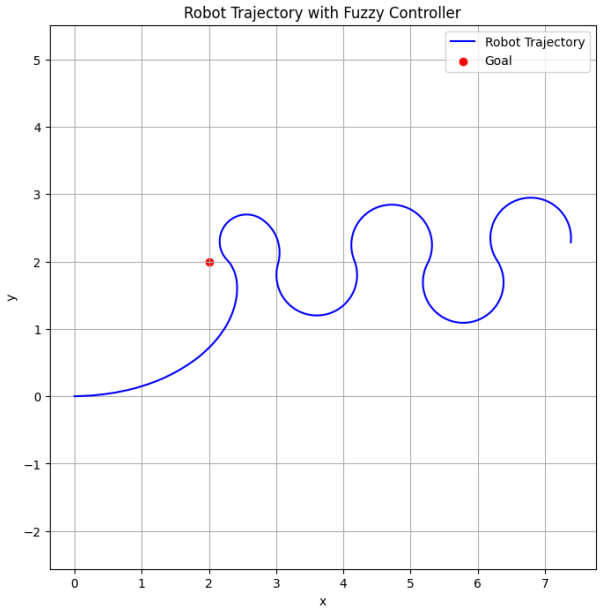
۳.طراحی کنترل کننده و رسم مسیر حرکت ربات در صفحه XY
برای تعریف توابع عضویت، ورودی ها به شکل small = [0 , 0 , 1.5] و Medium = [1 , 2 , 3] و Large = [2 , 2, 5] انجام داده ایم. و در تفاوت زاویه نیز این توابع به شکل small = [-pi/4 , 0 , pi/4] و medium = [-pi/2 , 0 , pi/2] و large = [-pi , -pi/2 , pi/2] تعریف شد اند. برای توابع عضویت خروجی نیز توابع عضویت در کد قابل مشاهده است.
قوانین فازی را به صورت زیر تعریف کرده ایم :

اگر فاصله به هدف کوچک باشد و تفاوت زاویه‌ای کوچک باشد، پس سرعت بالا و سرعت زاویه‌ای صفر باشد.

اگر فاصله به هدف زیاد باشد و تفاوت زاویه‌ای زیاد باشد، پس سرعت متوسط و سرعت زاویه‌ای متوسط باشد.

اگر فاصله به هدف زیاد باشد و تفاوت زاویه‌ای کوچک باشد، پس سرعت بالا و سرعت زاویه‌ای کم باشد.

در ادامه با استنتاج فازی میزان عضویت هر ورودی را اندازه گیری کرده ایم و برای بخش غیرفازی سازی، از غیرفازی ساز cetroid استفاده شده است.
در انتها مسیر حرکت ربات در صفحه رسم شده است :



سوال هفتم

ابتدا به کمک دستور gdown طبق آموزش ها، دادگان را دانلود کرده ایم. سپس در قسمت preprocess به پیش پردازش داده ها پرداخته ایم و اعمالی مانند نرمال سازی، حذف یا اصلاح دادگان و قرار دادن آن در data frame انجام شده است.
در مواردی، اگر داده ای موجود نبود آن را با میانگین همان ستون جمع کرده ایم.

```
df['Glucose'].replace(0, df['Glucose'].mean(), inplace=True)
df['BloodPressure'].replace(0, df['BloodPressure'].mean(), inplace=True)
df['SkinThickness'].replace(0, df['SkinThickness'].mean(), inplace=True)
df['Insulin'].replace(0, df['Insulin'].mean(), inplace=True)
df['BMI'].replace(0, df['BMI'].mean(), inplace=True)
```

سپس برای طراحی مدل ANFIS ، داده ها را به اموزش و ارزیابی تقسیم کرده ایم (۸۰ در مقابل ۲۰).

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

نرمال سازی را به کمک min max scaling انجام داده ایم.

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

سپس ۱۰۰ مرتبه اموزش را انجام داده ایم. در انتها دقت خروجی را نسبت به داده های ارزیابی بدست میاوریم. نتایج نشان دهنده دقتی به اندازه ی ۷۴ درصد است.

```
# Defining a simple fuzzy logic-inspired model
```

```
class SimpleANFIS:
    def __init__(self):
        # Define a very basic fuzzy rule structure (just an example, not
full ANFIS logic)
        self.model = MLPClassifier(hidden_layer_sizes=(10, 5),
max_iter=1000, random_state=42)

    def fit(self, X, y):
        # For simplicity, using a basic MLP with fuzzification-like
behavior
        self.model.fit(X, y)

    def predict(self, X):
        return self.model.predict(X)

    def score(self, X, y):
        return accuracy_score(y, self.predict(X))
anfis_accuracy = anfis_model.score(X_test_scaled, y_test)
```

```
0.7402597402597403
```

در ادامه یک مدل پرسپترون چند لایه با دو لایه مخفی (۱۰ و ۵ نورون) طراحی کردیم و ۱۰۰۰ دور تکرار کردیم تا به بهترین وزن و بایاس دست پیدا کنیم.

```
mlp_model = MLPClassifier(hidden_layer_sizes=(10, 5), max_iter=1000,
random_state=42)
mlp_model.fit(X_train_scaled, y_train)
mlp_accuracy = mlp_model.score(X_test_scaled, y_test)
```

در ادامه به کمک یک جدول عملکرد دو روش را مقایسه کرده ایم.
با توجه به نتایج، برای تصمیمات بالینی که نیاز به تفسیر پذیری بیشتری دارند مدل ANFIS ، و برای تحلیل سریع و تحلیل داده های گسترده تر مدل Perceptron مناسب تر است.

Model	Accuracy	interpretability	Complexity	Training Time
ANFIS	۰/۷۴۰۲۵۹۷۴۰۲۵۹۷۴۰۳	Higher	Moderate	Longer
MLP	۰/۷۴۰۲۵۹۷۴۰۲۵۹۷۴۰۳	Lower	Moderate	Shorter